

Computing IV Section 204: Project Portfolio

Project Portfolio Spring 2024

Salwan Sabil

PS0 - Hello SFML
PS1 - LFSR/PhotoMagic
PS2 - Pythagoras Tree
PS3 - Sokoban
PS4 - Dynamic NBody
PS5 - DNA Alignment
PS6 - RandWriter
PS7 - Kronos Log Parsing

Time to complete: 18 hours

Contents

1	PS0 - Hello SFML	4
1.1	Discussion of Assignment	4
1.2	Key Concepts	4
1.3	Accomplishments	4
1.4	Output	5
1.5	Code	6
1.5.1	Makefile	6
1.5.2	main.cpp	6
2	PS1 - LFSR/PhotoMagic	9
2.1	Discussion of Assignment	9
2.2	Key Concepts	9
2.3	Accomplishments	9
2.4	Output	10
2.5	Code	11
2.5.1	Makefile	11
2.5.2	PhotoMagic.hpp	11
2.5.3	PhotoMagic.cpp	12
2.5.4	FibLFSR.hpp	13
2.5.5	FibLFSR.cpp	13
2.5.6	test.cpp	15
3	PS2 - Pythagoras Tree	17
3.1	Discussion of Assignment	17
3.2	Key Concepts	17
3.3	Accomplishments	17
3.4	Output	17
3.5	Code	18
3.5.1	Makefile	18
3.5.2	main.cpp	19
3.5.3	PTree.hpp	20
3.5.4	PTree.cpp	20
4	PS3 - Sokoban	23
4.1	Discussion of Assignment	23
4.2	Key Concepts	23
4.3	Accomplishments	24
4.4	Output	25

4.5	Code	26
4.5.1	Makefile	26
4.5.2	main.cpp	26
4.5.3	Sokoban.hpp	29
4.5.4	Sokoban.cpp	31
5	PS4 - Dynamic NBody	42
5.1	Discussion of Assignment	42
5.2	Key Concepts	42
5.3	Accomplishments	42
5.4	Output	43
5.5	Code	44
5.5.1	Makefile	44
5.5.2	main.cpp	45
5.5.3	CelestialBody.hpp	47
5.5.4	CelestialBody.cpp	49
5.5.5	Universe.hpp	50
5.5.6	Universe.cpp	52
5.5.7	test.cpp	55
6	PS5 - DNA Alignment	56
6.1	Discussion of Assignment	56
6.2	Key Concepts	56
6.3	Accomplishments	56
6.4	Output	57
6.5	Code	58
6.5.1	Makefile	58
6.5.2	main.cpp	58
6.5.3	EDistance.hpp	59
6.5.4	EDistance.cpp	60
6.5.5	test.cpp	62
7	PS6 - RandWriter	64
7.1	Discussion of Assignment	64
7.2	Key Concepts	64
7.3	Accomplishments	64
7.4	Code	66
7.4.1	Makefile	66
7.4.2	TextWriter.cpp	66
7.4.3	RandWriter.hpp	67

7.4.4	RandWriter.cpp	69
7.4.5	test.cpp	73
8	PS7 - Kronos Log Parser	77
8.1	Discussion of Assignment	77
8.2	Key Concepts	77
8.3	Accomplishments	77
8.4	Output	78
8.5	Code	79
8.5.1	Makefile	79
8.5.2	main.cpp	79

1 PS0 - Hello SFML

1.1 Discussion of Assignment

The main aim of this project was to establish the essential dependencies for future SFML projects and gain a basic understanding of SFML projects, particularly regarding interface interaction tasks such as configuring Window Size, applying an Image to a Texture, and assigning a Sprite to that Texture. Additionally, setting up a Linux directive (achieved through WSL) for SFML dependencies was required. Once these initial setups were completed, the objective of this PS Assignment was to implement a texture onto a sprite and enable cardinal movement using keyboard event inputs. This allowed users to maneuver the sprite using the WASD keys, simulating game-like movement within the project.

1.2 Key Concepts

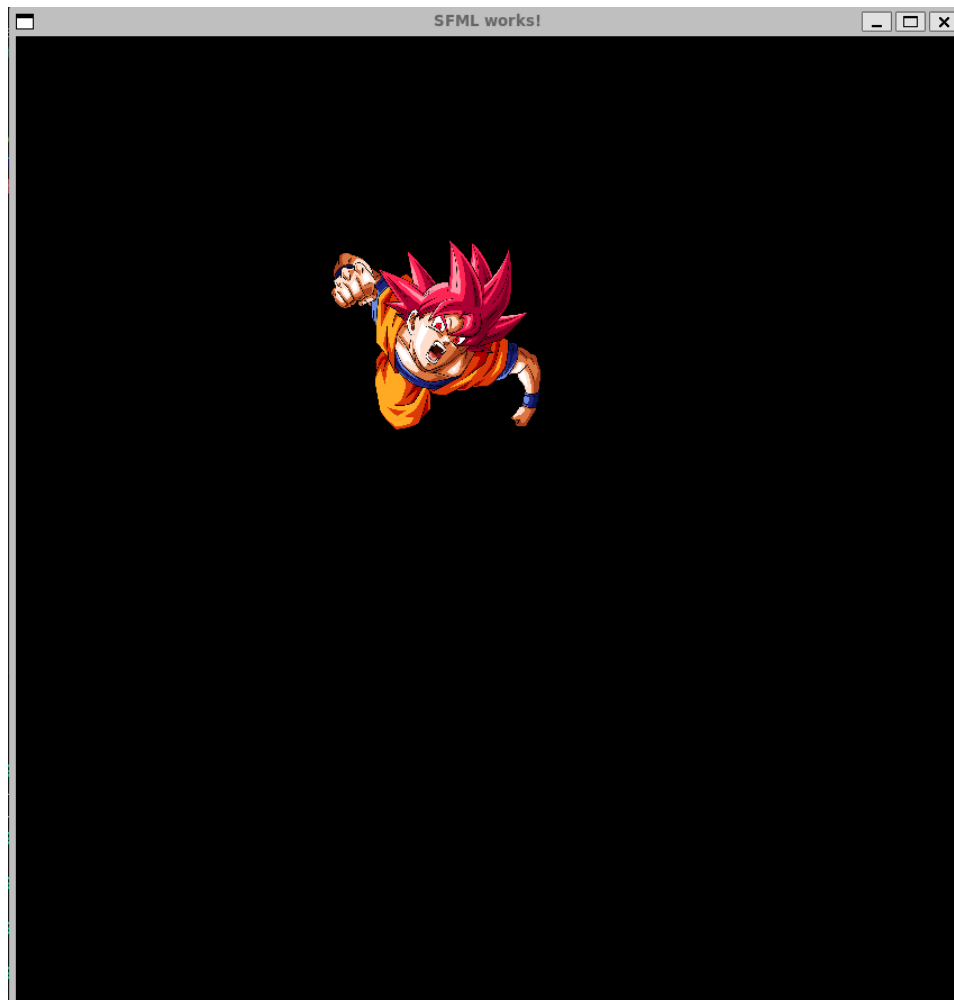
As outlined in the project overview, the main learnings from this assignment involved the practical application of SFML concepts like Sprites, Textures, Windows, and Keyboard Events. To grasp these concepts, I used the SFML documentation available on their website. This included understanding functions such as `'sf::Keyboard::isKeyPressed'` for detecting movement key presses. I also translated these key presses into position changes, which necessitated a clear understanding of SFML's coordinate system, where the origin (0,0) is located at the top left corner. My additional feature was to change the scaling. I did this since my sprite was so large. I set the 'D' key to decrease and 'I' to increase the image.

1.3 Accomplishments

Through this project, I gained insights into SFML's coordinate system and the manipulation of sprites, including the process of drawing a sprite. When a sprite is moved, what actually happens is that its current texture is erased and redrawn at a specified distance from its current position, based on the positional framework it occupies. It was crucial to understand that a sprite is essentially an image within the SFML window, bounded within a box without inherent boundary detection mechanisms.

1.4 Output

Figure 1: Sprite Image



1.5 Code

1.5.1 Makefile

```
1 CC = g++
2 CFLAGS = --std=c++17 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -
      -lsfml-system -lboost_unit_test_framework
4 # Your .hpp files
5 DEPS =
6 # Your compiled .o files
7 OBJECTS =
8 # The name of your program
9 PROGRAM = sfml-app
10
11 .PHONY: all clean lint
12
13 all: $(PROGRAM)
14
15 %.o: %.cpp $(DEPS)
16     $(CC) $(CFLAGS) -c $<
17
18 $(PROGRAM): main.o $(OBJECTS)
19     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
20
21 clean:
22     rm *.o $(PROGRAM)
23
24 lint:
25     cpplint *.cpp *.hpp
```

1.5.2 main.cpp

```
1 // Copyright [2023] <Copyright Salwan Sabil>
2 #include <SFML/Graphics.hpp>
3
4
5 int main() {
6     sf::RenderWindow window(sf::VideoMode(840, 859),
7                             "SFML works!");
7     sf::CircleShape shape(100.f);
```

```

8     shape.setFillColor(sf::Color::Green);
9
10 // Load a sprite to display
11     sf::Texture texture;
12     if (!texture.loadFromFile("sprite.png"))
13         return EXIT_FAILURE;
14     sf::Sprite sprite(texture);
15
16 // Start the game loop
17     while (window.isOpen()) {
18         // Process events
19         sf::Event event;
20
21         while (window.pollEvent(event)) {
22             // Close window: exit
23             if (event.type == sf::Event::Closed) {
24                 window.close();
25             }
26
27             if (sf::Keyboard::isKeyPressed(sf::
Keyboard:: Up)) {
28                 sprite.move(0, -10);
29             }
30
31             if (sf::Keyboard::isKeyPressed(sf::
Keyboard:: Down)) {
32                 sprite.move(0, 10);
33             }
34
35             if (sf::Keyboard::isKeyPressed(sf::
Keyboard:: Right)) {
36                 sprite.move(10, 0);
37             }
38
39             if (sf::Keyboard::isKeyPressed(sf::
Keyboard:: Left)) {
40                 sprite.move(-10, 0);
41             }
42
43             if (sf::Keyboard::isKeyPressed(sf::

```

```

44         Keyboard:: D)) {
45             sprite.setScale(0.2, 0.2);
46         }
47         if (sf::Keyboard::isKeyPressed(sf::
48             Keyboard:: I)) {
49             sprite.setScale(2, 2);
50         }
51     }
52     // Clear screen
53     window.clear();
54     // Draw the sprite
55     window.draw(sprite);
56     // Update the window
57     window.display();
58 }
59 return EXIT_SUCCESS;
60
61 return 0;
62 }

```

2 PS1 - LFSR/PhotoMagic

2.1 Discussion of Assignment

This project involved using Object-Oriented Programming (OOP) to address a particular problem. Specifically, I developed a Linear Feedback Shift Register class for encrypting an image using a designated seed value, which is provided to the Linear Feedback Shift Register. The decryption process relies on applying the same seed to the encrypted image to restore it to its original form. This encryption method involves XOR operations between each pixel of the image and seeded values. Also, this project introduced me to utilizing BOOST test cases to verify the functionality of the OOP implementation, ensuring that it operated as intended.

2.2 Key Concepts

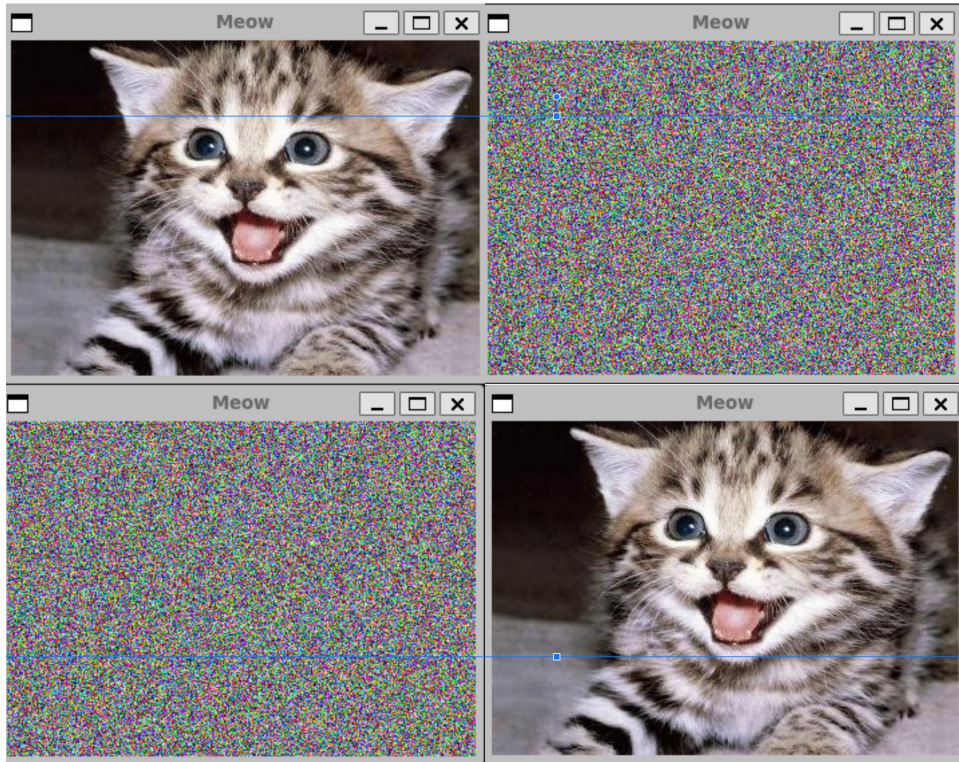
This project encompassed several key learning concepts, notably BOOST testing and understanding how to handle invalid arguments through throwing and catching exceptions in the constructor. It also involved revisiting bit manipulation concepts such as left-shifting and XOR operations with a bit list. Crucially, the project centered on implementing a vector to access the correct points within a 16-bit binary sequence.

2.3 Accomplishments

I refreshed my understanding of the significance of Unit Testing, particularly in OOP implementations. I've realized how important it is to extensively test the class created, as it may not work for future use cases initially and needs validation for its functionality. I learned how to pass command-line arguments to my object, enabling me to specify input/output files like .txt and .png, which is necessary for upcoming projects involving file handling. This also included working with multiple images in separate SFML windows and interacting with individual pixels, achieved through comprehending the linear feedback shift register to randomize pixel positions. To iterate through these pixels, I used nested for loops, iterating through columns one row at a time.

2.4 Output

Figure 2: Depicting the Encryption process of making an image static



2.5 Code

2.5.1 Makefile

```
1 CC = g++
2 CFLAGS = --std=c++17 -Wall -Werror -pedantic -g
3 LIBS = -lsfml-graphics -lsfml-audio -lsfml-window -
        lsfml-system -lboost_unit_test_framework
4 # Your .hpp files
5 DEPS = FibLFSR.hpp PhotoMagic.hpp
6 # Your compiled .o files
7 OBJS = FibLFSR.o PhotoMagic.o
8 # The name of your program
9
10 %.o: %.cpp $(DEPS)
11     $(CC) $(CFLAGS) -c $<
12
13 .PHONY: all clean lint
14
15 all: PhotoMagic test PhotoMagic.a
16
17 PhotoMagic.a: FibLFSR.o PhotoMagic.o
18     ar rcs PhotoMagic.a FibLFSR.o PhotoMagic.o
19
20 PhotoMagic: main.o $(OBJS)
21     $(CC) $(CFLAGS) -o $@ $^ $(LIBS)
22
23 test: $(OBJS) test.o
24     $(CC) $(CFLAGS) -o test $^ $(LIBS)
25
26 clean:
27     rm *.o PhotoMagic test PhotoMagic.a
28
29 lint:
30     cpplint *.cpp *.hpp
```

2.5.2 PhotoMagic.hpp

```
1 // Copyright [2023] <Salwan Sabil>
2 #pragma once
3 #include <SFML/Graphics.hpp>
```

```

4 #include "FibLFSR.hpp"
5
6 namespace PhotoMagic {
7 // Transforms image using FibLFSR
8 void transform(sf::Image&, FibLFSR*);
9 // Display an encrypted copy of the picture, using
    the LFSR to do the encryption
10 }

```

2.5.3 PhotoMagic.cpp

```

1 // Copyright [2024] <Salwan Sabil>
2 #include <iostream>
3 #include <SFML/System.hpp>
4 #include <SFML/Window.hpp>
5 #include <SFML/Graphics.hpp>
6 #include "FibLFSR.hpp"
7 #include "PhotoMagic.hpp"
8
9 namespace PhotoMagic {
10
11 void transform(sf::Image& image, FibLFSR* lfsr) {
12 sf::Vector2u imageSize = image.getSize();
13 sf::Color p;
14 int newPixel;
15
16 for (size_t x = 0; x< imageSize.x; x++) {
17 for (size_t y = 0; y< imageSize.y; y++) {
18 p = image.getPixel(x, y);
19 newPixel = lfsr ->generate(8);
20 p.r = p.r ^ newPixel;
21 newPixel = lfsr ->generate(8);
22 p.b = p.b ^ newPixel;
23 newPixel = lfsr ->generate(8);
24 p.g = p.g ^ newPixel;
25 image.setPixel(x, y, p);
26 }
27 }
28 }
29

```

```
30 } // namespace PhotoMagic
```

2.5.4 FibLFSR.hpp

```
1 // Copyright [2024] <Salwan Sabil>
2 #pragma once
3 #include <string>
4 #include <bitset>
5 #include <iostream>
6 #define fibonacci_limit 16
7
8 namespace PhotoMagic {
9 class FibLFSR {
10 public:
11 // Constructor to create LFSR with the given initial
    seed and tap
12 FibLFSR(std::string seed);
13 // Simulate one step and return the new bit as 0 or
    1
14 int step();
15
16 // Simulate k steps and return a k-bit integer
17 int generate(int k);
18
19 friend std::ostream& operator<<(std::ostream&, const
    FibLFSR& lfsr);
20 friend std::string conv_string(const FibLFSR& lfsr)
    ;
21
22 private:
23 // Any fields that you need
24 std::bitset <fibonacci_limit> list;
25 };
26
27 } // namespace PhotoMagic
```

2.5.5 FibLFSR.cpp

```
1 // Copyright [2024] <Salwan Sabil>
2 #include "FibLFSR.hpp"
3 #include <iostream>
```

```

4 #include <string>
5 #include <SFML/Window.hpp>
6 #include <SFML/System.hpp>
7 #include <SFML/Graphics.hpp>
8
9 #define fibonacci_limit 16
10
11 namespace PhotoMagic {
12 FibLFSR::FibLFSR(std::string seed) {
13 if (seed.size() != fibonacci_limit) {
14     throw std::invalid_argument("NOT 16 BITS");
15 }
16 std::bitset <fibonacci_limit> assign(seed);
17 list = assign;
18 }
19
20 int FibLFSR::step() {
21 bool exclusiveOR = list[15] ^ list[13] ^ list[12] ^
    list[10];
22 list <<= 1;
23 list[0] = exclusiveOR;
24 return exclusiveOR;
25 }
26
27 int FibLFSR::generate(int k) {
28     int result = 0;
29     int i = 0;
30
31     while (i < k) {
32         result = result * 2 + step();
33         i++;
34     }
35
36     return result;
37 }
38
39 std::string conv_string(const FibLFSR& lfsr) {
40     return lfsr.list.to_string();
41 }
42 std::ostream& operator<<(std::ostream& cout, const

```

```

        FibLFSR& lfsr) {
43     return cout << lfsr.list;
44 }
45
46 } // namespace PhotoMagic

```

2.5.6 test.cpp

```

1 // Copyright 2022
2 // By Dr. Rykalova
3 // Editted by Dr. Daly
4 // test.cpp for PS1a
5 // updated 1/8/2024
6
7 #include <iostream>
8 #include <string>
9
10 #include "FibLFSR.hpp"
11
12 #define BOOST_TEST_DYN_LINK
13 #define BOOST_TEST_MODULE Main
14 #include <boost/test/unit_test.hpp>
15
16 namespace PhotoMagic {
17 BOOST_AUTO_TEST_CASE(testStepInstr) {
18     FibLFSR l("1011011000110110");
19     BOOST_REQUIRE_EQUAL(l.step(), 0);
20     BOOST_REQUIRE_EQUAL(l.step(), 0);
21     BOOST_REQUIRE_EQUAL(l.step(), 0);
22     BOOST_REQUIRE_EQUAL(l.step(), 1);
23     BOOST_REQUIRE_EQUAL(l.step(), 1);
24     BOOST_REQUIRE_EQUAL(l.step(), 0);
25     BOOST_REQUIRE_EQUAL(l.step(), 0);
26     BOOST_REQUIRE_EQUAL(l.step(), 1);
27 }
28
29 BOOST_AUTO_TEST_CASE(testStepInstr2) {
30     FibLFSR l2("1011011000110110");
31     BOOST_REQUIRE_EQUAL(l2.generate(9), 51);
32 }

```

```

33
34 BOOST_AUTO_TEST_CASE(testSeed) {
35     FibLFSR l3("1011011000110110");
36     BOOST_REQUIRE_EQUAL(conv_string(l3), "
        1011011000110110");
37 }
38 BOOST_AUTO_TEST_CASE(testOutOfBoundsConstructor) {
39     FibLFSR l4("1011011000110110");
40     BOOST_REQUIRE_NO_THROW(l4.step());
41 }
42
43 BOOST_AUTO_TEST_CASE(testOutOfBoundsStep) {
44     FibLFSR l5("1011011000110110");
45     BOOST_REQUIRE_NO_THROW(l5.step());
46 }
47
48 BOOST_AUTO_TEST_CASE(testOutOfBoundsGenerate) {
49     FibLFSR l6("1011011000110110");
50     BOOST_REQUIRE_NO_THROW(l6.step());
51 }
52
53 } // namespace PhotoMagic

```

3 PS2 - Pythagoras Tree

3.1 Discussion of Assignment

The aim of this project was to create a pythagoras tree, constructed from squares and triangles. Users could customize the size and shape by specifying the number of recursions. To achieve this, I developed a PTree class capable of being drawn in SFML, recursively calling itself to reach the desired number of recursions. Additionally, this project involved working with SFML's drawable class to render the pythagoras tree correctly.

3.2 Key Concepts

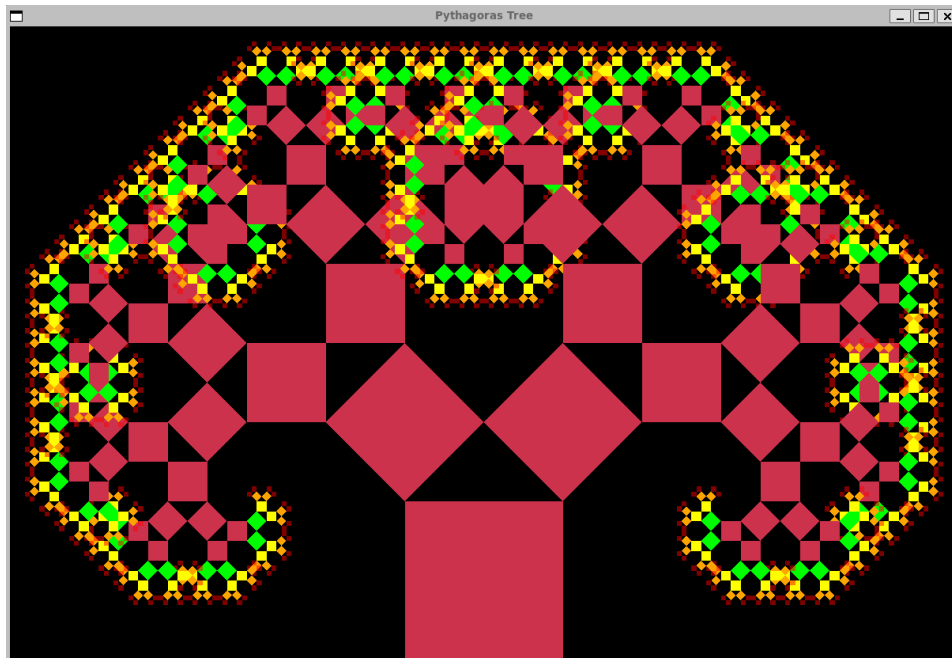
The primary concepts in this project centered on creating a drawable object class and its manipulation, alongside understanding SFML's coordinate system and recursion principles necessary for constructing the Pythagoras tree. Object recursion played a vital role in ensuring that the sub-squares retained the original properties, such as rotation, while keeping their size, which is adjusted with each recursion. This project provided a deep dive into object manipulation through OOP techniques, displaying their usefulness across various applications.

3.3 Accomplishments

From this assignment, I learned how to implement command line arguments so that the user will be able to control the depth and base of the pythagoras tree. My PTree class was able to use recursion in order to draw the squares for every level in the tree. The class also drew the tree out and set the position for each square. In my main file, I focused on opening the window and creating the instance for the tree. I was able to create the pTree() function which recursively draws the squares and then calls itself and draws two more squares. During the project, I faced several issues. I had immense trouble with the positioning of the base and depth which took a lot of trial and error to figure out. Once I completed the task, I decided to implement color in my tree as well for the bonus points.

3.4 Output

Figure 3: Pythagoras Tree with recursions



3.5 Code

3.5.1 Makefile

```
1 CC = g++
2 CFLAGS = -std=c++11 -g -Wall -Werror -pedantic -
  Wextra
3 LINT = cpplint
4
5 PTree: main.o PTree.o
6     $(CC) $(CFLAGS) -o PTree main.o PTree.o -lsfml-
  graphics -lsfml-window -lsfml-system
7
8 main.o: main.cpp PTree.hpp
9     $(CC) $(CFLAGS) -c -o main.o main.cpp
10
11 PTree.o: PTree.cpp PTree.hpp
12     $(CC) $(CFLAGS) -c -o PTree.o PTree.cpp
13
```

```

14 lint:
15     $(LINT) main.cpp PTree.cpp PTree.hpp
16
17 all: PTree lint
18
19 clean:
20     rm -f PTree main.o PTree.o

```

3.5.2 main.cpp

```

1 // Copyright [2024] <Salwan Sabil>
2 #include <iostream>
3 #include <SFML/Graphics.hpp>
4 #include "PTree.hpp"
5
6 int main(int, char* argv[]) {
7     int base = std::stoi(argv[1]);
8     int depth = std::stoi(argv[2]);
9
10    sf::RenderWindow window(sf::VideoMode(base * 6,
11        base * 4), "Pythagoras Tree");
12    sf::RectangleShape original{{static_cast<float>(
13        base),
14        static_cast<float>(base)}};
15    original.setPosition(base * 2.5, base * 3);
16    sf::Color color(sf::Color(205, 50, 76));
17
18    while (window.isOpen()) {
19        // Poll events
20        sf::Event event;
21        while (window.pollEvent(event)) {
22            // Close the window if the close event
23            // is triggered
24            if (event.type == sf::Event::Closed) {
25                window.close();
26                break;
27            }
28        }
29
30        window.clear();

```

```

28         PTree pTree(depth, original, window, color);
29         pTree.pTree();
30         window.display();
31     }
32
33     return 0;
34 }

```

3.5.3 PTree.hpp

```

1 // Copyright [2024] <Salwan Sabil>
2 #pragma once
3
4 #include <iostream>
5 #include <cmath>
6 #include <SFML/Graphics.hpp>
7
8 class PTree {
9 public:
10     PTree(int depth, sf::RectangleShape base,
11         sf::RenderTarget& window, sf::Color color);
12     void pTree();
13
14 private:
15     int depth;
16     sf::RectangleShape base;
17     sf::RenderTarget& window;
18     sf::Color color;
19 };

```

3.5.4 PTree.cpp

```

1 // Copyright [2024] <Salwan Sabil>
2 #include <iostream>
3 #include "PTree.hpp"
4
5 PTree::PTree(int depth, sf::RectangleShape base,
6 sf::RenderTarget& window, sf::Color color)
7 : depth(depth), base(base), window(window), color(
    color) {}
8

```

```

9 void PTree::pTree() {
10     base.setFillColor(color);
11     window.draw(base);
12
13     if (depth > 0) {
14         sf::RectangleShape left = base;
15         sf::RectangleShape right = base;
16
17         auto size = base.getSize();
18         auto offset = base.getTransform();
19         left.setSize(size * static_cast<float>((std
20             ::sqrt(2.0) / 2.0)));
21         right.setSize(size * static_cast<float>((std
22             ::sqrt(2.0) / 2.0)));
23         left.setOrigin(0, left.getSize().y);
24         right.setOrigin(right.getSize());
25         left.setPosition(offset.transformPoint({ 0,
26             0 }));
27         right.setPosition(offset.transformPoint({
28             size.x, 0 }));
29         left.rotate(-45);
30         right.rotate(45);
31
32         // Change the color based on the depth of
33         // the current iteration
34         switch (depth) {
35             case 4:
36                 color = sf::Color::Green;
37                 break;
38             case 3:
39                 color = sf::Color::Yellow;
40                 break;
41             case 2:
42                 color = sf::Color(255, 165, 0);
43                 break;
44             case 1:
45                 color = sf::Color(255, 0, 0, 128);
46                 break;
47             default:
48                 break;
49         }
50     }
51 }

```

```
44     }
45
46     // Recursively call the pTree function for
         the
47     // left rectangle with a decreased depth
48     PTree leftTree(depth - 1, left, window,
         color);
49     leftTree.pTree();
50     // Recursively call the pTree function for
         the
51     // right rectangle with a decreased depth
52     PTree rightTree(depth - 1, right, window,
         color);
53     rightTree.pTree();
54 }
55 }
```

4 PS3 - Sokoban

4.1 Discussion of Assignment

This project focuses on creating the Sokoban game, a block-pushing game where the player maneuvers blocks into designated spots. The core tasks involved developing a system capable of parsing information from a text file to generate playable levels. Players can move in cardinal directions (North, West, South, East) and push boxes in the same direction, provided no obstruction (another box or wall) exists adjacent to the box's destination. The game also includes a reset option, essential for situations where boxes become trapped in corners, preventing further movement. Victory conditions are met when all boxes occupy storage spaces or when all storage spaces are filled. In terms of SFML implementation, 64x64 pixel images serve as loaded sprites for each block, requiring inheritance from the Drawable class to override the draw() function effectively.

4.2 Key Concepts

One key concept was the handling of multiple textures and the method for drawing multiple elements. Overcoming challenges like defining boundaries for the character within the game and implementing conditions for movement constraints required meticulous planning and debugging as I progressed through the development stages. I gained experience in iterating row-wise and column-wise, essential for reading map information from a 2D text file. I designed the movePlayer function to locate the player '@' within the game level by iterating through it. The function calculates the new position of the player based on the specified direction as the function's argument. Following this a check is performed to verify the validity of the new player position. If the position is deemed invalid, the function exits without making any changes. In the event that the new position corresponds to a storage location, the function relocates the player to that position and sets the 'leavingStorageLocation' flag to true. If the player is departing from a storage location, the function updates the 'leavingStorageLocation' flag to true and records the locations of all storage locations. If the new position of the player is occupied by a box, the function assesses the validity of the new box position. Should the box position be deemed invalid, the function exits without taking any action. Conversely, if the new box position is valid, the function moves both the box and the player to their respective new positions. Resetting levels relied on re-reading the original text file to maintain consistency.

4.3 Accomplishments

One notable achievement was manipulating the pixel sizes. I learned more about sprites and SFML features, which helped me use different textures efficiently. I also learned how to optimally store the level data by overloading the `»` operator to read in a Sokoban level from a stream. I have a private member variable `char** grid` that dynamically allocates size depending on the level width and height. Another accomplishment was getting familiar with lambdas. I've enhanced the `loadTextures` function by incorporating a lambda function named `setTexture` to handle the assignment of textures to individual sprite objects. Within the lambda function, the texture data is loaded from the file specified by the provided filename using the `loadFromFile()` function of the `sf::Texture` class. If the loading process is successful, the lambda function assigns the texture to the sprite using the `setTexture()` function of the `sf::Sprite` class.

4.4 Output

Figure 4: Sokoban Game



4.5 Code

4.5.1 Makefile

```
1 CC=g++
2 CFLAGS=-Wall -Wextra -Werror -pedantic -std=c++17
3 LDFLAGS=-lsfml-graphics -lsfml-window -lsfml-system
   -lsfml-audio -lboost_unit_test_framework
4
5 all: Sokoban test Sokoban.a
6
7 Sokoban: main.o Sokoban.o
8     $(CC) $(CFLAGS) -o Sokoban main.o Sokoban.o $(
   LDFLAGS)
9
10 Sokoban.a: Sokoban.o main.o
11     ar rcs Sokoban.a Sokoban.o main.o
12
13 main.o: main.cpp
14     $(CC) $(CFLAGS) -c main.cpp
15
16 Sokoban.o: Sokoban.cpp
17     $(CC) $(CFLAGS) -c Sokoban.cpp
18
19 test: test.o Sokoban.o
20     $(CC) $(CFLAGS) -o test test.o Sokoban.o $(
   LDFLAGS)
21
22 test.o: test.cpp
23     $(CC) $(CFLAGS) -c test.cpp
24
25 lint:
26     cpplint --filter=-runtime/references,-build/c
   ++11 --root=. *.cpp *.h
27
28 clean:
29     rm -f *.o Sokoban test Sokoban.a
```

4.5.2 main.cpp

```
1 // Copyright [2024] <Salwan Sabil>
```

```

2 #include <SFML/Graphics.hpp>
3 #include "Sokoban.hpp"
4
5 int main(int argc, char* argv[]) {
6     SB::Sokoban sokoban;
7     bool gameWon = false;
8
9     if (argc != 2) {
10         std::cout << "Usage: " << argv[0] << " <
            filename>" << std::endl;
11         return 1;
12     }
13
14     std::string filename(argv[1]);
15
16     if (filename.substr(filename.size() - 4) != ".
        lvl") {
17         std::cout << "Error: Invalid file extension.
            Please specify a .lvl file." << std::
            endl;
18         return 1;
19     }
20
21     std::ifstream levelFile(filename);
22
23     if (!levelFile) {
24         std::cout << "Error: Failed to open level
            file " << filename << std::endl;
25         return 1;
26     }
27
28     sokoban.loadTextures();
29
30     levelFile >> sokoban;
31
32     sf::RenderWindow window(sf::VideoMode(sokoban.
        gridSize * sokoban.getScale(),
33
            sokoban.
                gridSize
                    *

```

```

sokoban
.
getScale
()), "
Sokoban
");

34
35 while (window.isOpen()) {
36     sf::Event event;
37     while (window.pollEvent(event)) {
38         if (event.type == sf::Event::Closed) {
39             window.close();
40         } else if (event.type == sf::Event::
            KeyPressed) {
41             if (!gameWon) {
42                 // Move the player in response
                    to keyboard events
43                 switch (event.key.code) {
44                     case sf::Keyboard::Up:
45                         sokoban.movePlayer(SB::
                            Direction::Up);
46                         break;
47                     case sf::Keyboard::Down:
48                         sokoban.movePlayer(SB::
                            Direction::Down);
49                         break;
50                     case sf::Keyboard::Left:
51                         sokoban.movePlayer(SB::
                            Direction::Left);
52                         break;
53                     case sf::Keyboard::Right:
54                         sokoban.movePlayer(SB::
                            Direction::Right);
55                         break;
56                     default:
57                         break;
58                 }
59                 window.clear();
60                 window.draw(sokoban);
61                 window.display();

```

```

62
63         if (sokoban.isWon()) {
64             gameWon = true;
65             window.clear();
66             window.draw(sokoban);
67             window.draw(sokoban.winText)
68             ;
69             window.display();
70             sokoban.playWinningSound();
71         }
72     } else if (event.key.code == sf::
Keyboard::R) {
73         // Restart the game when "R" key
is pressed
74         gameWon = false;
75         sokoban.resetGame();
76         window.clear();
77         window.draw(sokoban);
78         window.display();
79     }
80 }
81 if (!gameWon) {
82     window.draw(sokoban);
83     window.display();
84 }
85 }
86
87 return 0;
88 }

```

4.5.3 Sokoban.hpp

```

1 // Copyright [2024] <Salwan Sabil>
2 #pragma once
3
4 #include <iostream>
5 #include <fstream>
6 #include <string>
7 #include <vector>

```

```

8 #include <SFML/Graphics.hpp>
9 #include <SFML/Audio.hpp>
10
11 namespace SB {
12 enum class Direction { Up, Down, Right, Left };
13
14 class Sokoban : public sf::Drawable {
15 public:
16     Sokoban();
17     int width() const {return s_width;}
18     int height() const {return s_height;}
19     void loadTextures();
20     void movePlayer(Direction direction);
21     friend std::istream& operator>>(std::istream&
        stream, Sokoban& sokoban);
22     int gridSize;
23     sf::Sprite& getPlayer();
24     sf::Vector2i playerLoc() const;
25     void playWinningSound() {winning.play();}
26     void resetGame();
27 // used to set the texture of the player according
    to their direction
28     const sf::Texture& getTexture(Direction
        direction) const;
29
30 // public object of text to display win text in the
    main while loop
31     sf::Text winText;
32
33     bool isWon() const;
34     float getScale() const {return scale;}
35     int s_width;
36     int s_height;
37
38 private:
39     void draw(sf::RenderTarget& target, sf::
        RenderStates states) const override;
40     std::vector<sf::Sprite> storageLocations;
41     void setStorageLocations();
42

```

```

43     sf::Sprite environment;
44     sf::Sprite floor;
45     sf::Sprite player;
46     sf::Sprite storage;
47     sf::Sprite box;
48     sf::Sprite wall;
49     sf::Sprite playerUp;
50     sf::Sprite playerRight;
51     sf::Sprite playerLeft;
52     sf::Texture playerTexture;
53     sf::Texture floorTexture;
54     sf::Texture wallTexture;
55     sf::Texture boxTexture;
56     sf::Texture storageTexture;
57     sf::Texture environmentTexture;
58     sf::Texture playerUpTexture;
59     sf::Texture playerRightTexture;
60     sf::Texture playerLeftTexture;
61     sf::Texture texture;
62
63     sf::Sound winning;
64     sf::SoundBuffer buffer;
65
66     sf::Font font;
67     sf::Color color = sf::Color::Red;
68     float scale = 64.0;
69     char** grid;
70     char** originalGrid;
71 };
72 } // namespace SB

```

4.5.4 Sokoban.cpp

```

1 // Copyright [2024] <Salwan Sabil>
2 #include "Sokoban.hpp"
3 #include <iostream>
4
5 namespace SB {
6
7 Sokoban::Sokoban() {

```

```

8     loadTextures();
9 }
10
11 void Sokoban::movePlayer(Direction direction) {
12     if (isWon()) {
13         return;
14     }
15
16     int playerX = -1, playerY = -1;
17     bool foundPlayer = false;
18
19     for (int i = 0; i < gridSize; i++) {
20         for (int j = 0; j < gridSize; j++) {
21             if (grid[i][j] == '@') {
22                 playerX = j;
23                 playerY = i;
24                 foundPlayer = true;
25                 break;
26             }
27         }
28         if (foundPlayer) {
29             break;
30         }
31     }
32     int newPlayerX = playerX;
33     int newPlayerY = playerY;
34     switch (direction) {
35         case Direction::Up:
36             newPlayerY--;
37             break;
38         case Direction::Down:
39             newPlayerY++;
40             break;
41         case Direction::Left:
42             newPlayerX--;
43             break;
44         case Direction::Right:
45             newPlayerX++;
46             break;
47     }

```

```

48     if (newPlayerX < 0 || newPlayerX >= gridSize ||
49         newPlayerY < 0
50         || newPlayerY >= gridSize || grid[newPlayerY][
51             newPlayerX] == '#') {
52             return;
53         }
54     bool leavingStorageLocation = false;
55     if (grid[newPlayerY][newPlayerX] == '.') {
56         // Player is moving onto a storage location
57         grid[newPlayerY][newPlayerX] = '@';
58         grid[playerY][playerX] = '.';
59     } else if (grid[newPlayerY][newPlayerX] == 'a')
60     {
61         // Player is moving onto a storage location
62         grid[newPlayerY][newPlayerX] = '@';
63         grid[playerY][playerX] = '.';
64         leavingStorageLocation = true;
65     } else if (leavingStorageLocation) {
66         grid[playerY][playerX] = 'a';
67         setStorageLocations();
68     } else if (grid[newPlayerY][newPlayerX] == 'A')
69     {
70         // Player is moving a box onto a storage
71         // location
72         // Check if the new position of the box is
73         // valid
74         int newBoxX = newPlayerX + (newPlayerX -
75             playerX);
76         int newBoxY = newPlayerY + (newPlayerY -
77             playerY);
78         if (newBoxX < 0 || newBoxX >= gridSize ||
79             newBoxY < 0 || newBoxY >= gridSize ||
80             grid[newBoxY][newBoxX] == '#' || grid[
81                 newBoxY][newBoxX] == 'A'
82             || grid[newBoxY][newBoxX] == '1') {
83             return;
84         }
85         // Move the box to the new position
86         grid[newBoxY][newBoxX] = 'A';

```

```

79         grid[newPlayerY][newPlayerX] = '@';
80         if (grid[playerY][playerX] == '+') {
81             grid[playerY][playerX] = 'a';
82         } else {
83             grid[playerY][playerX] = '.';
84         }
85     } else if (grid[playerY][playerX] == '+') {
86         // Player is leaving a storage location
87         grid[playerY][playerX] = 'a';
88         leavingStorageLocation = true;
89     }
90
91     setStorageLocations();
92
93     // Update player sprite
94     player.setPosition(newPlayerX * scale,
95                        newPlayerY * scale);
96
97     // Update player texture based on direction
98     player.setTexture(getTexture(direction));
99     if (isWon()) {
100         winning.play();
101     }
102 }
103
104 std::istream& operator>>(std::istream& stream,
105 Sokoban& sokoban) {
106     int width, height;
107     stream >> width >> height;
108     sokoban.gridSize = width;
109     sokoban.grid = new char*[sokoban.gridSize];
110     sokoban.originalGrid = new char*[sokoban.
111                                     gridSize];
112     for (int i = 0; i < sokoban.gridSize; i++) {
113         sokoban.grid[i] = new char[sokoban.gridSize];
114         sokoban.originalGrid[i] = new char[sokoban.
115                                             gridSize];
116     }
117 }

```

```

114         for (int j = 0; j < sokoban.gridSize; j++) {
115             stream >> sokoban.grid[i][j];
116             sokoban.originalGrid[i][j] = sokoban.
                grid[i][j];
117         }
118     }
119     return stream;
120 }
121
122 void Sokoban::draw(sf::RenderTarget& target, sf::
    RenderStates states) const {
123     sf::Sprite environment = this->environment;
124     sf::Sprite wall = this->wall;
125     sf::Sprite player = this->player;
126     sf::Sprite box = this->box;
127     sf::Sprite storage = this->storage;
128     sf::Sprite floor = this->floor;
129     target.draw(environment, states);
130
131     // Draw floors
132     for (int i = 0; i < gridSize; i++) {
133         for (int j = 0; j < gridSize; j++) {
134             if (grid[i][j] == '.') {
135                 floor.setPosition(j * scale, i *
                    scale);
136                 target.draw(floor, states);
137             }
138         }
139     }
140
141     for (int i = 0; i < gridSize; i++) {
142         for (int j = 0; j < gridSize; j++) {
143             if (grid[i][j] == 'a') {
144                 storage.setPosition(j * scale, i *
                    scale);
145                 target.draw(storage, states);
146             } else if (grid[i][j] == '1') {
147                 box.setPosition(j * scale, i * scale
                    );
148                 target.draw(box, states);

```

```

149         }
150     }
151 }
152
153 // Draw storage
154 for (int i = 0; i < gridSize; i++) {
155     for (int j = 0; j < gridSize; j++) {
156         if (grid[i][j] == 'a') {
157             storage.setPosition(j * scale, i *
158                               scale);
159             target.draw(storage, states);
160         }
161     }
162
163 // Draw boxes
164 for (int i = 0; i < gridSize; i++) {
165     for (int j = 0; j < gridSize; j++) {
166         if (grid[i][j] == 'A') {
167             box.setPosition(j * scale, i * scale
168                             );
169             target.draw(box, states);
170         } else if (grid[i][j] == '1') {
171             box.setPosition(j * scale, i * scale
172                             );
173             target.draw(box, states);
174         }
175     }
176
177 // Draw player
178 for (int i = 0; i < gridSize; i++) {
179     for (int j = 0; j < gridSize; j++) {
180         if (grid[i][j] == '@') {
181             player.setPosition(j * scale, i *
182                               scale);
183             floor.setPosition(j * scale, i *
184                               scale);
185             target.draw(floor, states);
186             target.draw(player, states);

```

```

184         }
185     }
186 }
187
188 // Draw walls
189 for (int i = 0; i < gridSize; i++) {
190     for (int j = 0; j < gridSize; j++) {
191         if (grid[i][j] == '#') {
192             wall.setPosition(j * scale, i *
                             scale);
193             target.draw(wall, states);
194         }
195     }
196 }
197 }
198
199 sf::Sprite& Sokoban::getPlayer() {
200     return player;
201 }
202
203 // function to set the player texture based on
    direction
204 const sf::Texture& Sokoban::getTexture(Direction
    direction) const {
205     switch (direction) {
206         case Direction::Up:
207             return playerUpTexture;
208         case Direction::Left:
209             return playerLeftTexture;
210             break;
211         case Direction::Right:
212             return playerRightTexture;
213             break;
214         default:
215             return playerTexture;
216             break;
217     }
218 }
219
220 void Sokoban::loadTextures() {

```

```

221     // lambda function to load textures and set them
        for a given sprite
222     auto setTexture = [](sf::Sprite& sprite,
223     sf::Texture& texture, const std::string&
        filename) {
224     if (texture.loadFromFile(filename)) {
225         sprite.setTexture(texture);
226     }
227 };
228
229     // Load textures and set them for each sprite
230     setTexture(environment, environmentTexture, "
        environment_03.png");
231     setTexture(floor, floorTexture, "ground_01.png")
        ;
232     setTexture(wall, wallTexture, "block_06.png");
233     setTexture(box, boxTexture, "crate_03.png");
234     setTexture(player, playerTexture, "player_05.png
        ");
235     setTexture(storage, storageTexture, "ground_04.
        png");
236     setTexture(playerUp, playerUpTexture, "player_08
        .png");
237     setTexture(playerRight, playerRightTexture, "
        player_17.png");
238     setTexture(playerLeft, playerLeftTexture, "
        player_20.png");
239
240 // Load sound buffer
241     if (buffer.loadFromFile("winning.wav")) {
242         winning.setBuffer(buffer);
243     }
244
245     // Load font
246     if (font.loadFromFile("arial.ttf")) {
247         winText.setFont(font);
248     }
249
250     // Set properties for winText
251     winText.setString("You win! Press R to Restart.")

```

```

        );
252     winText.setCharacterSize(24);
253     winText.setFillColor(color);
254     winText.setPosition(10, 10);
255 }
256
257
258 sf::Vector2i Sokoban::playerLoc() const {
259     for (int i = 0; i < gridSize; i++) {
260         for (int j = 0; j < gridSize; j++) {
261             if (grid[i][j] == '@') {
262                 return sf::Vector2i(j, i);
263             }
264         }
265     }
266     return sf::Vector2i(-1, -1); // if location
        not found
267 }
268
269 void Sokoban::setStorageLocations() {
270     // Clear storage locations
271     storageLocations.clear();
272
273     // Update storage locations based on original
        grid
274     for (int i = 0; i < gridSize; i++) {
275         for (int j = 0; j < gridSize; j++) {
276             if (originalGrid[i][j] == 'a') {
277                 if (grid[i][j] == '.') {
278                     grid[i][j] = 'a';
279                     storage.setTexture(
                        storageTexture);
280                     storage.setPosition(j * scale, i
                        * scale);
281                     storageLocations.push_back(
                        storage);
282                 } else if (grid[i][j] == 'a') {
283                     storage.setTexture(
                        storageTexture);
284                     storage.setPosition(j * scale, i

```

```

                * scale);
285         storageLocations.push_back(
                storage);
286     }
287 }
288 }
289 }
290 }
291
292 bool Sokoban::isWon() const {
293     int boxesOnStorage = 0;
294     int storageAreas = 0;
295     int boxes = 0;
296
297     for (int i = 0; i < gridSize; i++) {
298         for (int j = 0; j < gridSize; j++) {
299             if (originalGrid[i][j] == 'a') {
300                 storageAreas++;
301                 if (grid[i][j] == 'A') {
302                     boxesOnStorage++;
303                 }
304             }
305             if (grid[i][j] == 'A') {
306                 boxes++;
307             }
308         }
309     }
310
311     if (boxes > storageAreas) {
312         return boxesOnStorage == storageAreas;
313     } else {
314         return boxesOnStorage == boxes;
315     }
316 }
317
318 void Sokoban::resetGame() {
319     // Restore the grid to its original state
320     for (int i = 0; i < gridSize; i++) {
321         for (int j = 0; j < gridSize; j++) {
322             grid[i][j] = originalGrid[i][j];

```

```

323     }
324 }
325     // Update player position to the starting
        position
326 sf::Vector2i startingPosition = playerLoc();
327 if (startingPosition.x != -1 && startingPosition
    .y != -1) {
328     player.setPosition(startingPosition.x *
        scale, startingPosition.y * scale);
329 }
330 // Clear storage locations
331 storageLocations.clear();
332 // Update storage locations based on the
        original grid
333 setStorageLocations();
334 }
335 } // namespace SB

```

5 PS4 - Dynamic NBody

5.1 Discussion of Assignment

The objective of this task was to develop a simulation of a universe featuring the initial four planets in our solar system. In this simulation, the planets were programmed to orbit the sun according to specific physics calculations, which considered factors such as velocity, position, mass, and more. This was accomplished using SFML's window system and involved converting viewport coordinates to an xy plane or coordinate grid format.

5.2 Key Concepts

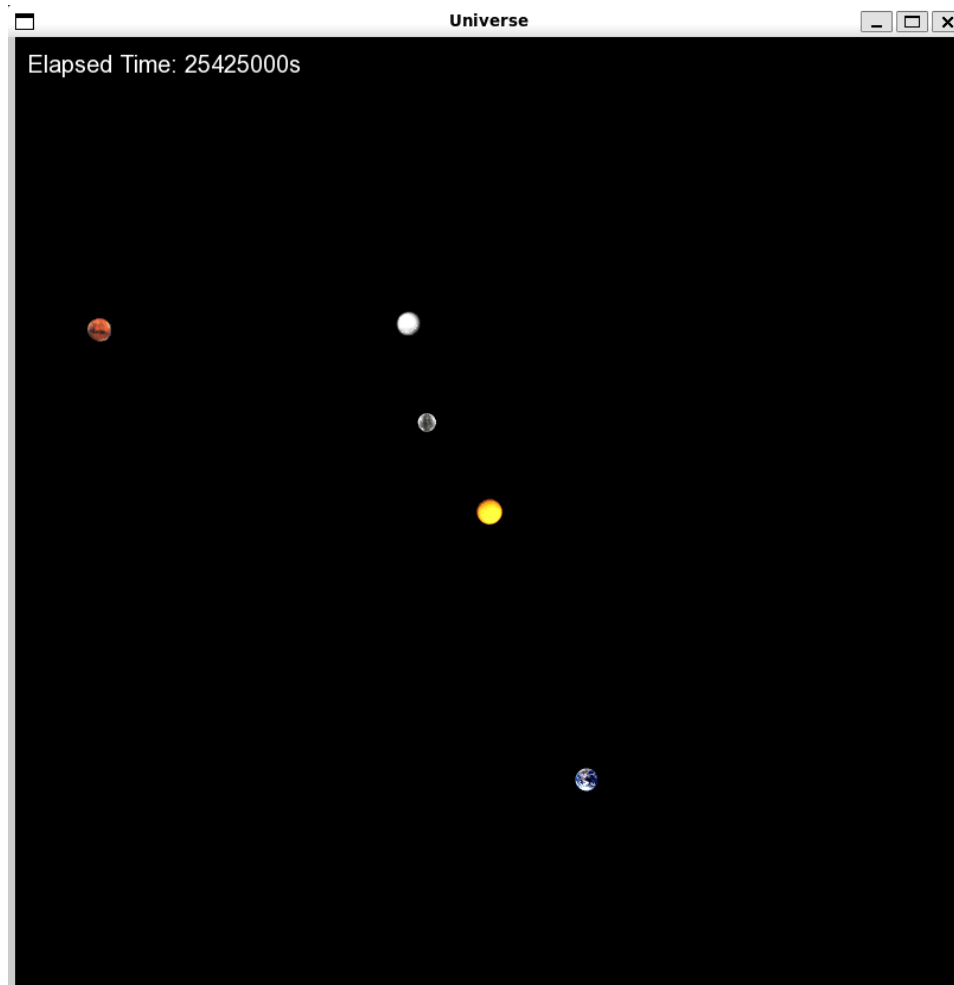
The project loads and display the planets in the solar system. The input is a text file which includes information about individual planets. Using this I had to determine the scale of the drawing window. I had to read how many rows of body information given by n. I used setters and getters for my Universe and CelestialBody. I also created a vector for the planets in my program.

5.3 Accomplishments

Some major decisions I made was overloading the input stream operator » for both Universe and CelestialBody as well as the output stream operator « for the Universe and CelestialBody. Another feature is reading input from a file. Also, I implemented the elapsed time with units (seconds) for extra credit.

5.4 Output

Figure 5: Active simulation after steps



5.5 Code

5.5.1 Makefile

```
1 CC = g++
2 CFLAGS = -std=c++17 -Wall -Wextra -Werror -pedantic
3 SFML_LIBS = -lsfml-graphics -lsfml-window -lsfml-
    system -lboost_unit_test_framework
4
5 all: NBody test
6
7 NBody: main.o CelestialBody.o Universe.o
8     $(CC) $(CFLAGS) -o NBody main.o CelestialBody.o
    Universe.o $(SFML_LIBS)
9
10 main.o: main.cpp Universe.hpp CelestialBody.hpp
11     $(CC) $(CFLAGS) -c main.cpp
12
13 CelestialBody.o: CelestialBody.cpp CelestialBody.hpp
14     $(CC) $(CFLAGS) -c CelestialBody.cpp
15
16 Universe.o: Universe.cpp Universe.hpp CelestialBody.
   hpp
17     $(CC) $(CFLAGS) -c Universe.cpp
18
19 NBody.a: CelestialBody.o Universe.o
20     ar rcs NBody.a CelestialBody.o Universe.o
21
22 test: test.o CelestialBody.o Universe.o NBody.a
23     $(CC) $(CFLAGS) -o test test.o CelestialBody.o
    Universe.o $(SFML_LIBS)
24
25 test.o: test.cpp Universe.hpp CelestialBody.hpp
26     $(CC) $(CFLAGS) -c test.cpp
27
28 lint:
29     cpplint --filter=-runtime/references,-build/c
    ++11 --root=. *.cpp *.h
30
31 clean:
```

```
32      rm -f NBody test main.o CelestialBody.o Universe
        .o NBody.a test.o
```

5.5.2 main.cpp

```
1 // Copyright [2024] <Salwan Sabil>
2 #include <iostream>
3 #include <string>
4 #include "SFML/Window.hpp"
5 #include "SFML/Graphics.hpp"
6 #include "Universe.hpp"
7 #include "CelestialBody.hpp"
8
9 int main(int, const char* argv[]) {
10     double totalTime = std::stod(argv[1]);
11     double secondsPerStep = std::stod(argv[2]);
12     NB::Universe universe;
13
14     std::cin >> universe;
15
16     int windowSizeX = 800;
17     int windowSizeY = 800;
18
19     for (int i = 0; i < universe.getBodies(); i++) {
20         auto temp = universe.getPlanet(i);
21         temp->setWindowDimensions(windowSizeX,
22                                 windowSizeY);
23         temp->setPosition(temp->getXPos(), temp->
24                           getYPos());
25     }
26
27     sf::RenderWindow window(sf::VideoMode(
28         windowSizeX, windowSizeY), "Universe");
29     double timeElapsed = 0;
30     window.setFramerateLimit(60);
31     sf::Font font;
32
33     if (!font.loadFromFile("arial.ttf")) {
34         std::cout << "Can't open font file" << std::
35         endl;
```

```

32         return -1;
33     }
34     sf::Text text;
35     text.setFont(font);
36     text.setCharacterSize(20);
37     text.setFillColor(sf::Color::White);
38     text.setPosition(10, 10);
39
40     while (window.isOpen()) {
41         sf::Event event;
42         while (window.pollEvent(event)) {
43             if (event.type == sf::Event::Closed ||
44                 timeElapsed > totalTime) {
45                 window.close();
46             }
47
48             std::string Time = "Elapsed Time: " + std::
49                 to_string(static_cast<int>(timeElapsed))
50                 + "s";
51             text.setString(Time);
52             window.clear();
53
54             for (int i = 0; i < universe.getBodies(); i
55                 ++){
56                 window.draw(*universe.getPlanet(i));
57             }
58
59             window.draw(text);
60
61             window.display();
62             universe.step(secondsPerStep);
63             if (timeElapsed > totalTime) {
64                 secondsPerStep = 0;
65             }
66             timeElapsed += secondsPerStep;
67     }
68     std::cout << universe;
69     return 0;
70 }

```

5.5.3 CelestialBody.hpp

```
1 // Copyright [2024] <Salwan Sabil>
2 #pragma once
3
4 #include <iostream>
5 #include <string>
6 #include <SFML/Graphics.hpp>
7
8 namespace NB {
9 class CelestialBody : public sf::Drawable {
10 public:
11     CelestialBody(double x, double y, double xVel,
12                   double yVel,
13                   double mass, const std::string& planetName);
14
15     double getXPos() const {
16         return xPos;
17     }
18
19     double getYPos() const {
20         return yPos;
21     }
22
23     double getXVelocity() const {
24         return xVelocity;
25     }
26
27     double getYVelocity() const {
28         return yVelocity;
29     }
30
31     void setVel(double x, double y) {
32         xVelocity = x;
33         yVelocity = y;
34     }
35
36     double getMass() const {
37         return Mass;
```

```

38
39     sf::Vector2f getPos() const {
40         return sf::Vector2f(xPos, yPos);
41     }
42
43     std::string getFilename() const {
44         return PlanetName;
45     }
46
47     void setRadius(double rad) {
48         radius = rad;
49     }
50
51     void setForce(double F) {
52         force = F;
53     }
54
55     double getForce() const {
56         return force;
57     }
58
59     void setAccel(double accX , double accY) {
60         accelX = accX;
61         accelY = accY;
62     }
63
64     double getAccelX() const {
65         return accelX;
66     }
67
68     double getAccelY() const {
69         return accelY;
70     }
71
72     void setWindowDimensions(int width, int depth) {
73         windowX = width;
74         windowY = depth;
75     }
76
77     void setPosition(double x, double y);

```

```

78
79 private:
80     int windowX, windowY;
81     double xPos, yPos, xVelocity, yVelocity, Mass,
        radius, force, accelX, accelY;
82     std::string PlanetName;
83     sf::Image planetIMG;
84     sf::Texture planetTexture;
85     sf::Sprite Planet;
86
87     virtual void draw(sf::RenderTarget& target, sf::
        RenderStates states) const {
88         target.draw(Planet, states);
89     }
90 };
91 } // namespace NB

```

5.5.4 CelestialBody.cpp

```

1 // Copyright [2024] <Salwan Sabil>
2 #include "CelestialBody.hpp"
3
4 namespace NB {
5 CelestialBody::CelestialBody(double x, double y,
    double xVel, double yVel,
6
    double mass, const std
        ::string& planetName
    ) {
7     xPos = x;
8     yPos = y;
9     xVelocity = xVel;
10    yVelocity = yVel;
11    Mass = mass;
12    PlanetName = planetName;
13
14    if (!planetIMG.loadFromFile(planetName)) {
15        std::cout << "Failed to load the specified
            file" << std::endl;
16    }
17

```

```

18     planetTexture.loadFromImage(planetIMG);
19     Planet.setTexture(planetTexture);
20 }
21
22 void CelestialBody::setPosition(double x, double y)
23 {
24     xPos = x;
25     yPos = y;
26
27     double viewportX = (x / radius) * (windowX / 2)
28         + (windowX / 2);
29     double viewportY = (-y / radius) * (windowY / 2)
30         + (windowY / 2);
31
32     double viewXPos = viewportX;
33     double viewYPos = viewportY;
34
35     Planet.setOrigin(Planet.getTextured->getSize().
36         x / 2, Planet.getTextured->getSize().y / 2);
37     Planet.setPosition(viewXPos, viewYPos);
38 }
39 } // namespace NB

```

5.5.5 Universe.hpp

```

1 // Copyright [2024] <Salwan Sabil>
2 #pragma once
3
4 #include <iostream>
5 #include <iomanip>
6 #include <memory>
7 #include "CelestialBody.hpp"
8
9 namespace NB {
10 class Universe {
11 public:
12     Universe() = default;
13
14     int getSize() {
15         return universeContents.size ();

```

```

16     }
17
18     int getBodies() const {
19         return numCelestialBodies;
20     }
21
22     double getRad() const {
23         return uniRadius;
24     }
25
26     std::shared_ptr<CelestialBody> getPlanet(int x)
27     {
28         return universeContents[x];
29     }
30
31     friend std::istream& operator>>(std::istream& in
32         , Universe& uni);
33     friend std::ostream& operator<<(std::ostream&
34         out, Universe& uni);
35
36     void printContents();
37
38     void step(double seconds);
39
40     double calcForce(CelestialBody &anchor,
41         CelestialBody &source);
42
43     double getXDistance(CelestialBody &anchor,
44         CelestialBody &source);
45
46     double getYDistance(CelestialBody &anchor,
47         CelestialBody &source);
48
49     double getRDistance(CelestialBody &anchor ,
50         CelestialBody &source);
51
52 private:
53     int numCelestialBodies;
54     double uniRadius;

```

```

49     std::vector<std::shared_ptr<CelestialBody>>
        universeContents;
50 };
51 } // namespace NB

```

5.5.6 Universe.cpp

```

1 // Copyright [2024] <Salwan Sabil>
2 #include <cmath>
3 #include "Universe.hpp"
4 #define G 6.674e-11
5
6 namespace NB {
7 std::istream& operator>>(std::istream& in, Universe&
    uni) {
8     double x, y, xVel, yVel, mass;
9     std::string planetName;
10    in >> uni.numCelestialBodies;
11    in >> uni.uniRadius;
12    int i = 0;
13    while (i < uni.numCelestialBodies) {
14        in >> x >> y >> xVel >> yVel >> mass >>
            planetName;
15        auto temp = std::make_shared<CelestialBody>(
            x, y, xVel, yVel, mass, planetName);
16        temp->setRadius(uni.uniRadius);
17        uni.universeContents.push_back(temp);
18        i++;
19    }
20    return in;
21 }
22
23 std::ostream& operator<<(std::ostream& out, Universe
    & uni) {
24    out << uni.getBodies() << std::endl;
25    out << uni.getRad() << std::endl;
26    for (int i = 0; i < uni.getBodies(); i++) {
27        auto planet = uni.getPlanet(i);
28        out << std::setw(14) << planet->getXPos()
29            << std::setw(14) << planet->getYPos()

```

```

30         << std::setw(14) << planet->getXVelocity
           ()
31         << std::setw(14) << planet->getYVelocity
           ()
32         << std::setw(14) << planet->getMass()
33         << std::setw(20) << planet->getFilename
           () << std::endl;
34     }
35     return out;
36 }
37
38 double Universe::getXDistance(CelestialBody &anchor,
    CelestialBody &source) {
39     return (source.getXPos() - anchor.getXPos());
40 }
41
42 double Universe::getYDistance(CelestialBody &anchor,
    CelestialBody &source) {
43     return (source.getYPos() - anchor.getYPos());
44 }
45
46 double Universe::getRDistance(CelestialBody &anchor
    , CelestialBody &source)
47 { double xDist , yDist , rDist;
48     xDist = getXDistance(anchor , source);
49     yDist = getYDistance(anchor , source);
50     rDist = sqrt((xDist* xDist) + (yDist* yDist));
51     return rDist;
52 }
53
54 double Universe::calcForce(CelestialBody &anchor,
    CelestialBody &source) {
55     double rDist = getRDistance(anchor , source);
56     double force = ((G* ((anchor.getMass()* source.
        getMass())) / (rDist* rDist)));
57     return force;
58 }
59
60 void Universe::step(double seconds) {
61     for (int i = 0; i < numCelestialBodies ; i++) {

```

```

62     double accelX , accelY , uptXVelo = 0,
63     uptYVelo = 0, force, xDist, yDist, rDist,
64     mass;
65     double initXVelo = getPlanet(i) -> getXVelocity
66         (),
67     initYVelo = getPlanet(i) -> getYVelocity(),
68     forceX = 0 , forceY = 0;
69     mass = getPlanet(i) -> getMass();
70     for (int j = 0; j < numCelestialBodies; j++) {
71         if (j != i) {
72             force = calcForce(*getPlanet(i), *
73                             getPlanet(j));
74             xDist = getXDistance(*getPlanet(i), *
75                                 getPlanet(j));
76             yDist = getYDistance(*getPlanet(i), *
77                                 getPlanet(j));
78             rDist = getRDistance(*getPlanet(i), *
79                                 getPlanet(j));
80             forceX += force * (xDist / rDist);
81             forceY += force * (yDist / rDist);
82         }
83     }
84     accelX = forceX / mass;
85     accelY = forceY / mass;
86     getPlanet(i) -> setAccel(accelX, accelY);
87     uptXVelo = initXVelo + (seconds * accelX);
88     uptYVelo = initYVelo + (seconds * accelY);
89     getPlanet(i) -> setVel(uptXVelo, uptYVelo);
90 }
91 }
92 } // namespace NB

```

5.5.7 test.cpp

```
1 // Copyright [2024] <Salwan Sabil>
2 #define BOOST_TEST_DYN_LINK
3 #define BOOST_TEST_MODULE NBodyTests
4 #include <boost/test/unit_test.hpp>
5 #include "Universe.hpp"
6 #include "CelestialBody.hpp"
7
8 namespace NB {
9 BOOST_AUTO_TEST_CASE(test_universe_num_bodies) {
10 Universe universe;
11 BOOST_CHECK_EQUAL(universe.getBodies(), 0);
12 // Check if the initial number of bodies is 0
13 }
14
15 BOOST_AUTO_TEST_CASE(test_celestial_body_position) {
16 CelestialBody body(10.0, 20.0, 30.0, 40.0, 50.0, "
    Earth ");
17 BOOST_CHECK_CLOSE(body.getXPos(), 10.0, 1e-6);
18 // Check if X position is approximately 10.0
19 BOOST_CHECK_CLOSE(body.getYPos(), 20.0, 1e-6);
20 // Check if Y position is approximately 20.0
21 BOOST_CHECK_CLOSE(body.getXVelocity(), 30.0, 1e-6);
22 // Check if X velocity is approximately 30.0
23 BOOST_CHECK_CLOSE(body.getYVelocity(), 40.0, 1e-6);
24 // Check if Y velocity is approximately 40.0
25 BOOST_CHECK_CLOSE(body.getMass(), 50.0, 1e-6);
26 // Check if Mass is approximately 50.0
27 BOOST_CHECK_EQUAL(body.getFilename(), " Earth ");
28 // Check if the filename is " Earth "
29 }
30
31 BOOST_AUTO_TEST_CASE(test_universe_radius ) {
32 Universe universe;
33 BOOST_CHECK_CLOSE(universe.getRad(), 0.0, 1e-6);
34 }
35 } // namespace NB
```

6 PS5 - DNA Alignment

6.1 Discussion of Assignment

This project involves developing a program to calculate the optimal sequence alignment of two DNA strings, introducing concepts from computational biology and dynamic programming. Additionally, the project will utilize valgrind for memory analysis and will measure and report the implementation's space and time performance.

6.2 Key Concepts

For my implementation, I chose to use dynamic programming to calculate the edit distance between two genetic sequences, utilizing the 2D vector method to store the optimal edit distances between the input strings. This decision was based on dynamic programming's efficiency and its ability to handle larger inputs more effectively compared to recursive methods. I incorporated a lambda expression in my program. This lambda function is designed to return the minimum value between two integer inputs. The expression `minlambda(minlambda(a, b), c)` is utilized to first find the minimum between a and b using `minlambda(a, b)`, and then determine the minimum between that result and c using `minlambda(minlambda(a, b), c)`. Consequently, the final result is the minimum value among the three integers.

6.3 Accomplishments

I came across another practical use of 2D arrays with a more intricate runtime, and I gained a deeper understanding of dynamic programming. Essentially, dynamic programming involves breaking down a problem into smaller parts and solving it step by step. I understand there are several different methods for dynamically solving such problems. This experience reinforced the importance of efficient memory management, especially when dealing with substantial data volumes in projects. I am also now familiar with running valgrind and stepping through errors.

6.4 Output

Figure 6: DNA Sequence

```
ssabil@LAPTOP-B89257RH:/mnt/c/COMP IV/homeworks/ps5$ ./EDistance < example10.txt
Edit distance = 7
A T 1
A A 0
C - 2
A A 0
G G 0
T G 1
T T 0
A - 2
C C 0
C A 1

Execution time is 0.000386 seconds
Edit distance = 7
ssabil@LAPTOP-B89257RH:/mnt/c/COMP IV/homeworks/ps5$
```

6.5 Code

6.5.1 Makefile

```
1 CC = g++
2 CFLAGS = --std=c++17 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -
      -lsfml-system -lboost_unit_test_framework
4 DEPS = EDistance.hpp
5 OBJECTS = EDistance.o
6 PROGRAM = EDistance
7 LIBRARY = EDistance.a
8
9 .PHONY: all clean lint
10
11 all: $(PROGRAM) $(LIBRARY) test
12
13 %.o: %.cpp $(DEPS)
14     $(CC) $(CFLAGS) -c $<
15
16 $(PROGRAM): main.o $(OBJECTS)
17     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
18
19 $(LIBRARY): $(OBJECTS)
20     ar rcs $@ $^
21
22 test: test.o $(OBJECTS) $(LIBRARY)
23     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
24
25 clean:
26     rm -f *.o $(PROGRAM) $(LIBRARY) test
27
28 lint:
29     cpplint *.cpp *.hpp
```

6.5.2 main.cpp

```
1 // Copyright [2024] <Salwan Sabil>
2
3 #include <stdint.h>
4 #include <iostream>
```

```

5 #include <SFML/System.hpp> // To use sf::Clock and
   sf::Time classes for execution time
6 #include "EDistance.hpp"
7
8 int main() {
9     sf::Clock clock; // Create a clock object to
   measure execution time
10    sf::Time t;
11    std::string s1;
12    std::string s2;
13    std::cin >> s1; // Read the first sequence from
   standard input
14    std::cin >> s2; // Read the second sequence
   from standard input
15    // Create an EDistance object with the two
   sequences
16    EDistance run(s1, s2);
17    int distance = run.optDistance();
18    // Print results according to instructions
19    std::cout << "Edit distance = " << distance <<
   std::endl;
20    std::cout << run.alignment() << std::endl;
21    t = clock.getElapsedTime();
22    std::cout << "Execution time is " << t.asSeconds
   () << " seconds \n";
23    std::cout << "Edit distance = " << distance <<
   std::endl;
24    return 0;
25 }

```

6.5.3 EDistance.hpp

```

1 // Copyright [2024] <Salwan Sabil>
2 #pragma once
3
4 #include <iostream>
5 #include <sstream>
6 #include <string>
7 #include <vector>
8 #include <algorithm>

```

```

9
10 class EDistance {
11 public:
12     // A constructor that accepts the two strings to
        be compared,
13     // and that allocates any data structures
        necessary in order to do the work
14     EDistance(std::string x, std::string y);
15     // A static method int penalty(char a, char b)
        that
16     // returns the penalty for aligning chars a and
        b
17     static int penalty(char a, char b);
18     // A static method int min3(int a, int b, int c)
        which
19     // returns the minimum of the three arguments.
20     static int min(int a, int b, int c);
21     // A method int optDistance() which
22     // populates the matrix based on having the two
        strings,
23     // and returns the optimal distance
24     int optDistance();
25     // A method string alignment() which traces the
        matrix and returns a string
26     // that can be printed to display the actual
        alignment.
27     std::string alignment();
28     // 2D vector to store the optimal edit distances
        of the two strings
29     std::vector<std::vector<int>> opt;
30     std::string s1;
31     std::string s2;
32 private:
33 };

```

6.5.4 EDistance.cpp

```

1 // Copyright [2024] <Salwan Sabil>
2
3 #include <iostream>

```

```

4 #include <sstream>
5 #include <string>
6 #include <vector>
7 #include <algorithm>
8 #include "EDistance.hpp"
9
10 EDistance::EDistance(std::string a, std::string b) {
11     s1 = a;
12     s2 = b;
13     std::vector<int> MatrixVec(s1.size() + 1, 0);
14     for (size_t x = 0; x < s2.size() + 1; x++) {
15         opt.push_back(MatrixVec);
16     }
17 }
18
19 int EDistance::min(int a, int b, int c) {
20     auto min_lambda = [](int x, int y) { return x <
        y ? x : y; };
21     return min_lambda(min_lambda(a, b), c);
22 }
23
24 int EDistance::penalty(char a, char b) {
25     if (a == b) {
26         return 0;
27     } else {
28         return 1;
29     }
30 }
31
32 std::string EDistance::alignment() {
33     std::stringstream align;
34     size_t i = 0;
35     size_t j = 0;
36     while (i < s2.length() || j < s1.length()) {
37         if (i < s2.length() && j < s1.length() &&
38             opt[i][j] == opt[i+1][j+1] + penalty(s1[
39                 j], s2[i])) {
40             align << s1[j] << " " << s2[i] << " "
                << penalty(s1[j], s2[i]) << std:::
                endl;

```

```

41         i++;
42         j++;
43     } else if (i < s2.length() && opt[i][j] ==
44               opt[i+1][j] + 2) {
45         align << "- " << s2[i] << " " <<
46               penalty(s1[j], s2[i])+1 << std::endl
47               ;
48         i++;
49     } else {
50         align << s1[j] << " - " <<
51               penalty(s1[j], s2[i])+1 << std::endl
52               ;
53         j++;
54     }
55 }
56
57 int EDistance::optDistance() {
58     for (size_t i = 0; i < s2.length() + 1; i++) {
59         opt[i][s1.length()] = 2 * (s2.length() - i);
60     }
61     for (size_t j = 0; j < s1.length() + 1; j++) {
62         opt[s2.length()][j] = 2 * (s1.length() - j);
63     }
64     for (size_t i = s2.length()- 1; i != static_cast
65           <size_t>(-1); i--) {
66         for (size_t j = s1.length()- 1; j !=
67               static_cast<size_t>(-1); j--) {
68             opt[i][j] = min(opt[i+1][j+1] + penalty(
69                   s1[j], s2[i]),
70                   opt[i+1][j] + 2, opt[i][
71                   j+1] + 2);
72         }
73     }
74     return opt[0][0];
75 }

```

6.5.5 test.cpp

```

1 // Copyright [2024] <Salwan Sabil>
2 #define BOOST_TEST_MODULE EDistanceTests
3 #include <boost/test/included/unit_test.hpp>
4 #include "EDistance.hpp"
5
6 BOOST_AUTO_TEST_CASE(testPenaltyEqualCharacters) {
7     EDistance ed("abc", "abc");
8     BOOST_CHECK_EQUAL(ed.penalty('a', 'a'), 0);
9     BOOST_CHECK_EQUAL(ed.penalty('b', 'b'), 0);
10    BOOST_CHECK_EQUAL(ed.penalty('c', 'c'), 0);
11 }
12
13 BOOST_AUTO_TEST_CASE(testPenaltyDiffCharacters) {
14     EDistance ed("abc", "xyz");
15     BOOST_CHECK_EQUAL(ed.penalty('a', 'x'), 1);
16     BOOST_CHECK_EQUAL(ed.penalty('b', 'y'), 1);
17     BOOST_CHECK_EQUAL(ed.penalty('c', 'z'), 1);
18 }
19
20 BOOST_AUTO_TEST_CASE(testMin) {
21     EDistance ed("", "");
22     BOOST_CHECK_EQUAL(ed.min(1, 2, 3), 1);
23     BOOST_CHECK_EQUAL(ed.min(2, 1, 3), 1);
24     BOOST_CHECK_EQUAL(ed.min(3, 2, 1), 1);
25 }
26
27 BOOST_AUTO_TEST_CASE(testOptDistance) {
28     EDistance ed("kitten", "sitting");
29     BOOST_CHECK_EQUAL(ed.optDistance(), 4);
30 }

```

7 PS6 - RandWriter

7.1 Discussion of Assignment

For this project, I created a random text generation program called "RandWriter." This program creates text by analyzing a provided input text using a Markov model with a specified order of k . The implementation involved creating the RandWriter class and a corresponding client program named TextWriter.cpp. To test the functionality, I used Boost Unit testing.

7.2 Key Concepts

For this project the central algorithm was the implementation of a Markov model of order k , which uses k -grams and their frequencies to generate random text. I used maps to store k -grams and $k+1$ -grams frequencies. I also used set to store the unique characters in the input text. Constructor: Initializes the RandWriter object with the input text and order k . orderK(): Returns the order k of the Markov model. freq(): Returns the frequency of a given k -gram or k -gram with a specific character appended. kRand(): Returns a random character based on the input k -gram's frequency distribution. generate(): Generates a random text of length L using the input k -gram as a seed. I created test cases for RandWriter class methods, including error handling. These are the exceptions I used: I threw `std::runtimeerror` when the input k -gram was not of the correct length or when there were no valid k -grams for random character generation. I used `BOOST_REQUIRE_THROW` and `BOOST_REQUIRE_NO_THROW` to check for the proper throwing of exceptions in various scenarios. In test.cpp, I utilized a lambda expression to display k -grams alongside their respective frequencies. Additionally, I made a new function called 'forEachKgram' within the RandWriter class, which accepts a lambda function as a parameter. This lambda function is designed to take a k -gram and its frequency as inputs, enabling it to print these values accordingly.

7.3 Accomplishments

In this project, I focused on implementing a Markov model of order k to generate random text using k -grams and their frequencies. I utilized maps to store k -grams and their frequencies, along with sets for unique characters. The RandWriter class managed essential functions like initializing the object, determining the order k , calculating frequencies, generating random characters, and producing random text. Testing was necessary, involving

error handling for incorrect inputs and exception validation using `BOOST-REQUIRETHROW` and `BOOSTREQUIRENOTHROW`. The `test.cpp` file showcased k-grams and their frequencies using a lambda expression, and I added a `'forEachKgram'` function to dynamically print k-grams and their frequencies. Overall, the project emphasized algorithm design, thorough testing, and effective error handling.

7.4 Code

7.4.1 Makefile

```
1 CXX = g++
2 CXXFLAGS = -std=c++17 -Wall -Wextra -Werror -
    pedantic -g
3 LDFLAGS = -lboost_unit_test_framework
4
5 all: TextWriter test TextWriter.a
6
7 TextWriter: TextWriter.o RandWriter.o
8     $(CXX) $(CXXFLAGS) -o TextWriter TextWriter.o
    RandWriter.o
9
10 TextWriter.o: TextWriter.cpp RandWriter.hpp
11     $(CXX) $(CXXFLAGS) -c TextWriter.cpp
12
13 RandWriter.o: RandWriter.cpp RandWriter.hpp
14     $(CXX) $(CXXFLAGS) -c RandWriter.cpp
15
16 test: test.o RandWriter.o
17     $(CXX) $(CXXFLAGS) -o test test.o RandWriter.o $
    (LDFLAGS)
18
19 test.o: test.cpp RandWriter.hpp
20     $(CXX) $(CXXFLAGS) -c test.cpp
21
22 TextWriter.a: TextWriter.o RandWriter.o
23     ar rcs TextWriter.a TextWriter.o RandWriter.o
24
25 lint:
26     cpplint *.cpp *.hpp
27
28 clean:
29     rm -f TextWriter test TextWriter.a *.o
```

7.4.2 TextWriter.cpp

```
1 // Copyright [2024] <Salwan Sabil>
2
```

```

3 #include <iostream>
4 #include "RandWriter.hpp"
5
6 int main(int argc, char* argv[]) {
7     // Input validation: checking if correct number
      of command-line arg is given
8     if (argc != 3) {
9         std::cerr << "Usage: " << argv[0] << " k L"
          << std::endl;
10        return 1;
11    }
12
13    // Parse command-line arguments for k and L
      values
14    int k = std::stoi(argv[1]);
15    int L = std::stoi(argv[2]);
16
17    // Read input text from standard input
18    std::string input_text((std::istreambuf_iterator
      <char>
19    (std::cin)), std::istreambuf_iterator<char>());
20
21    // Create a RandWriter object using the input
      text and k value
22    RandWriter rand_writer(input_text, k);
23
24    // Generate a random text using the RandWriter
      object
25    std::string kgram = input_text.substr(0, k);
26    std::string generated_text = rand_writer.
      generate(kgram, L);
27
28    // Output the generated text
29    std::cout << generated_text << std::endl;
30
31    return 0;
32 }

```

7.4.3 RandWriter.hpp

```

1 // Copyright [2024] <Salwan Sabil>
2
3 #pragma once
4 #include <iostream>
5 #include <map>
6 #include <set>
7 #include <string>
8
9 class RandWriter {
10 public:
11     // Create a Markov model of order k from given
        text
12     // Assume that text has length at least k
13     RandWriter(const std::string text, size_t k);
14     size_t orderK() const; // Order k of Markov
        model
15
16     // Number of occurrences of kgram in text
17     // Throw an exception if kgram is not length k
18     int freq(const std::string& kgram) const;
19     // Number of times that character c follows
        kgram
20     // if order=0, return num of times that char c
        appears
21     // (throw an exception if kgram is not of length
        k)
22     int freq(const std::string& kgram, char c) const
        ;
23
24     // Random character following given kgram
25     // (throw an exception if kgram is not length k)
26     // (throw an exception if no such kgram)
27     char kRand(const std::string& kgram);
28
29     // Generate a string of length L characters by
        simulating a trajectory
30     // through the corresponding Markov chain. The
        first k characters of
31     // the newly generated string should be the
        argument kgram

```

```

32     // Throw an exception if kgram is not of length
        k
33     // Assume that L is at least k
34     std::string generate(const std::string& kgram,
        size_t L);
35
36     // Overload the stream insertion operator << and
        display the internal state
37     // of the Markov model. Print out the order,
        alphabet, and the frequencies
38     // of the k-grams and k+1-grams
39
40     friend std::ostream &operator<<(std::ostream &os
        ,
41     const RandWriter &rw);
42
43     // Member variables
44     int k_;
45     std::string text_;
46     std::set<char> alphabet_;
47     std::map<std::string, int> kgram_freq_;
48     std::map<std::string, int> kplus1_freq_;
49
50     // Calls lambda function for each k-gram
        frequency in the model.
51     template <typename Func>
52         void forEachKgram(Func func) const {
53         for (const auto &entry : kgram_freq_) {
54             func(entry.first, entry.second);
55         }
56 }
57
58 private:
59 };

```

7.4.4 RandWriter.cpp

```

1 // Copyright [2024] <Salwan Sabil>
2
3 #include "RandWriter.hpp"

```

```

4 #include <stdexcept>
5 #include <random>
6 #include <algorithm>
7 #include <vector>
8
9 RandWriter::RandWriter(const std::string text,
    size_t k) : k_(k), text_(text) {
10     for (const auto& c : text) {
11         alphabet_.insert(c);
12     }
13
14     // Calculate the k-gram frequencies
15     for (size_t i = 0; i < text.size(); i++) {
16         std::string kgram;
17         for (size_t j = 0; j < k; j++) {
18             kgram += text[(i + j) % text.size()];
19         }
20         kgram_freq_[kgram]++;
21     }
22
23     // Calculate the k+1-gram frequencies
24     for (size_t i = 0; i < text.size(); i++) {
25         std::string kplus1gram;
26         for (size_t j = 0; j < k + 1; j++) {
27             kplus1gram += text[(i + j) % text.size()
28                 ];
29         }
30         kplus1_freq_[kplus1gram]++;
31     }
32
33 // orderK() method: Returns the order k of the
    Markov model
34 size_t RandWriter::orderK() const {
35     return k_;
36 }
37
38 // number of occurrences of kgram in text
39 int RandWriter::freq(const std::string& kgram) const
    {

```

```

40     if (kgram.size() != static_cast<std::string::
        size_type>(k_)) {
41         throw std::runtime_error("kgram is not of
            length k");
42     }
43     auto it = kgram_freq_.find(kgram);
44     if (it == kgram_freq_.end()) {
45         return 0;
46     }
47     return it->second;
48 }
49
50 // freq() method (overloaded): Returns the frequency
    of a given
51 // k-gram with a specific character appended.
52 int RandWriter::freq(const std::string& kgram, char
    c) const {
53     if (kgram.size() != static_cast<std::string::
        size_type>(k_)) {
54         throw std::runtime_error("kgram is not of
            length k");
55     }
56     if (alphabet_.find(c) == alphabet_.end()) {
57         throw std::runtime_error("Character not in
            alphabet");
58     }
59     std::string kplus1gram = kgram + c;
60     auto it = kplus1_freq_.find(kplus1gram);
61     if (it == kplus1_freq_.end()) {
62         return 0;
63     }
64     return it->second;
65 }
66
67 // Returns random char based on the input k-gram's
    frequency distribution.
68 char RandWriter::kRand(const std::string& kgram) {
69     if (kgram.size() != static_cast<std::string::
        size_type>(k_)) {
70         throw std::runtime_error("kgram is not of

```

```

        length k");
71     }
72     std::vector<char> candidates;
73     std::vector<int> weights;
74
75     for (const auto& c : alphabet_) {
76         int freq_c = freq(kgram, c);
77         if (freq_c > 0) {
78             candidates.push_back(c);
79             weights.push_back(freq_c);
80         }
81     }
82
83     if (candidates.empty()) {
84         throw std::runtime_error("No such kgram");
85     }
86
87     std::random_device rd;
88     std::mt19937 gen(rd());
89     std::discrete_distribution<> dist(weights.begin
        (), weights.end());
90
91     return candidates[dist(gen)];
92 }
93
94 // Generates a random text of length L using the
    input k-gram as a seed.
95 std::string RandWriter::generate(const std::string&
    kgram, size_t L) {
96     if (kgram.size() != static_cast<std::string::
    size_type>(k_)) {
97         throw std::runtime_error("kgram is not of
        length k");
98     }
99     if (L < static_cast<size_t>(k_)) {
100     throw std::runtime_error("L is less than k");
101     }
102
103     std::string result = kgram;
104     std::string current_kgram = kgram;

```

```

105
106     for (size_t i = 0; i < L - k_; i++) {
107         char next_char = kRand(current_kgram);
108         result.push_back(next_char);
109         current_kgram.erase(0, 1);
110         current_kgram.push_back(next_char);
111     }
112
113     return result;
114 }
115
116 // overload the stream insertion operator and
117 // display
118 // the internal state of the Markov Model
119 std::ostream &operator<<(std::ostream &os, const
120 RandWriter &rw) {
121     os << "Order: " << rw.k_ << std::endl;
122     os << "Alphabet: ";
123     for (const auto &c : rw.alphabet_) {
124         os << c;
125     }
126     os << std::endl;
127
128     os << "k-grams:" << std::endl;
129     for (const auto &kv : rw.kgram_freq_) {
130         os << kv.first << ": " << kv.second << std::
131         endl;
132     }
133
134     os << "k+1-grams:" << std::endl;
135     for (const auto &kv : rw.kplus1_freq_) {
136         os << kv.first << ": " << kv.second << std::
137         endl;
138     }
139
140     return os;
141 }

```

7.4.5 test.cpp

```

1 // Copyright [2024] <Salwan Sabil>
2
3 #define BOOST_TEST_MODULE RandWriterTest
4 #include <stdexcept>
5 #include <boost/test/included/unit_test.hpp>
6 #include "RandWriter.hpp"
7
8 // Test case for checking frequency counts of k-
   grams and k+1-grams
9 BOOST_AUTO_TEST_CASE(RandWriter_frequency_test) {
10     std::string input_text = "gagggagaggcgagaaa";
11     int k = 2;
12
13     RandWriter rw(input_text, k);
14
15     // Lambda function to print k-gram and its
       frequency
16     rw.forEachKgram([](const std::string &kgram, int
       frequency) {
17         std::cout << kgram << ": " << frequency <<
       std::endl;
18     });
19     // Print k-gram frequencies
20     for (const auto &entry : rw.kgram_freq_) {
21         std::cout << entry.first << ": " << entry.
       second << std::endl;
22     }
23
24     std::cout << std::endl;
25
26     // Print k+1-gram frequencies
27     for (const auto &entry : rw.kplus1_freq_) {
28         std::cout << entry.first << ": " << entry.
       second << std::endl;
29     }
30 }
31
32 // Test case for testing RandWriter class methods,
   including error handling.
33 BOOST_AUTO_TEST_CASE(RandWriter_test) {

```

```

34     std::string input_text = "gagggagaggcgagaaa";
35     int k = 2;
36
37     RandWriter rw(input_text, k);
38     // Test orderK() method
39     BOOST_REQUIRE_EQUAL(rw.orderK(), k);
40
41     // Test freq() method with single k-gram
42     parameter
43     BOOST_REQUIRE_EQUAL(rw.freq("ga"), 5);
44     BOOST_REQUIRE_EQUAL(rw.freq("gg"), 3);
45
46     // Test error handling for incorrect k-gram size
47     BOOST_REQUIRE_THROW(rw.freq("gaa"), std::
48         runtime_error);
49     BOOST_REQUIRE_NO_THROW(rw.freq("ga"));
50
51     // Test freq() method with k-gram and character
52     parameters
53     BOOST_REQUIRE_EQUAL(rw.freq("ga", 'g'), 4);
54     BOOST_REQUIRE_EQUAL(rw.freq("ga", 'a'), 1);
55
56     // Test error handling for invalid character
57     BOOST_REQUIRE_THROW(rw.freq("ga", 'b'), std::
58         runtime_error);
59     BOOST_REQUIRE_NO_THROW(rw.freq("ga", 'a'));
60
61     // Test kRand() method
62     char random_char = rw.kRand("ga");
63     BOOST_REQUIRE(random_char == 'a' || random_char
64         == 'g');
65     BOOST_REQUIRE_THROW(rw.kRand("gaa"), std::
66         runtime_error);
67
68     // Test generate() method
69     std::string generated_text = rw.generate("ga",
70         10);
71     BOOST_REQUIRE_EQUAL(generated_text.substr(0, k),
72         "ga");
73     BOOST_REQUIRE_EQUAL(generated_text.length(), 10)

```

```
        ;  
66     BOOST_REQUIRE_THROW(rw.generate("gaa", 10), std  
        ::runtime_error);  
67 }
```

8 PS7 - Kronos Log Parser

8.1 Discussion of Assignment

This project focuses on Kronos InTouch device logs, aiming to scan through them to create a report file that logs boot successes and failures. The logs include a start sequence that contains a date and time along with the boot time, which must be recorded. This task required implementing Regex functions to extract the necessary information accurately.

8.2 Key Concepts

I used regular expressions to extract specific details from the log file and organized them into suitable data structures. I utilized the Boost C++ libraries to handle dates and times. I created variables to monitor the program's state as it processed each line of the input. Specifically, I utilized a boolean variable to determine if a boot sequence was ongoing and employed two `boost::posixtime::ptime` variables to record the start and end times of each boot event. Lastly, I implemented `std::ofstream` to create an output file containing the necessary header and boot information.

8.3 Accomplishments

In this project, I successfully utilized regular expressions to extract specific details from log files, ensuring accurate data retrieval. Additionally, I integrated Boost C++ libraries to effectively handle dates and times, enhancing the precision of my program. I also created and managed variables to monitor the program's state as it processed each line of input. Finally, I learned how to implement `std::ofstream`, completing the logging process seamlessly. These accomplishments demonstrate a structured and efficient approach to handling log data, utilizing advanced C++ features for effective processing and output generation.

8.4 Output

Figure 7: Boot Times Report for Device 1

```
1 Device Boot Report
2
3 InTouch log file: device1_intouch.log
4 Lines Scanned: 443838
5
6 === Device Boot ===
7 435369(device1_intouch.log): 2014-03-25 19:11:59 Boot Start
8 435759(device1_intouch.log): 2014-03-25 19:15:02 Boot Completed
9 | Boot Time: 183000ms
10
11 === Device Boot ===
12 436500(device1_intouch.log): 2014-03-25 19:29:59 Boot Start
13 436859(device1_intouch.log): 2014-03-25 19:32:44 Boot Completed
14 | Boot Time: 165000ms
15
16 === Device Boot ===
17 440719(device1_intouch.log): 2014-03-25 22:01:46 Boot Start
18 440791(device1_intouch.log): 2014-03-25 22:04:27 Boot Completed
19 | Boot Time: 161000ms
20
21 === Device Boot ===
22 440866(device1_intouch.log): 2014-03-26 12:47:42 Boot Start
23 441216(device1_intouch.log): 2014-03-26 12:50:29 Boot Completed
24 | Boot Time: 167000ms
25
26 === Device Boot ===
27 442094(device1_intouch.log): 2014-03-26 20:41:34 Boot Start
28 442432(device1_intouch.log): 2014-03-26 20:44:13 Boot Completed
29 | Boot Time: 159000ms
30
31 === Device Boot ===
32 443073(device1_intouch.log): 2014-03-27 14:09:01 Boot Start
33 443411(device1_intouch.log): 2014-03-27 14:11:42 Boot Completed
34 | Boot Time: 161000ms
```

8.5 Code

8.5.1 Makefile

```
1 CXX = g++
2 CXXFLAGS = -std=c++17 -Wall -Wextra -Werror -
    pedantic
3 LDFLAGS = -lboost_regex -lboost_date_time
4
5 all: ps7
6
7 ps7: main.o
8     g++ main.o -o ps7 -l boost_regex -Wall -Werror -
    pedantic
9
10 main.o: main.cpp
11     g++ -c main.cpp -Wall -Werror -pedantic
12
13 lint:
14     cpplint *.cpp
15
16 clean:
17     rm *.o ps7
```

8.5.2 main.cpp

```
1 // Copyright [2024] <Salwan Sabil>
2
3 #include <iostream>
4 #include <fstream>
5 #include <string>
6 #include <boost/regex.hpp>
7 #include <boost/date_time/gregorian/gregorian.hpp>
8 #include <boost/date_time/posix_time/posix_time.hpp>
9
10 int main(int argc, const char* argv[]) {
11     std::ifstream file(argv[1]);
12     if (file.fail()) {
13         std::cout << "Could not open file" << std
            ::endl;
14         exit(1);
15     }
```

```

15     }
16
17     std::string outFile(argv[1]);
18     outFile += ".rpt";
19     std::ofstream output_file(outFile);
20
21     // First pass: count total number of lines
22     int totalLines = 0;
23     std::string line;
24     while (std::getline(file, line)) {
25         totalLines += 1;
26     }
27     file.clear();
28     file.seekg(0);
29
30     // Header info
31     output_file << "Device Boot Report\n\n";
32     output_file << "InTouch log file: " << argv[1]
33         << "\n";
34     output_file << "Lines Scanned: " << totalLines
35         << "\n\n";
36
37     // Second pass: process lines and generate
38     // output
39     boost::regex startup
40         ("([0-9]+)-([0-9]+)-([0-9]+) ([0-9]+):([0-9]+)
41          :([0-9]+): \\(log.c.166\\) server started.*"
42          );
43     boost::regex success
44         ("([0-9]+)-([0-9]+)-([0-9]+) ([0-9]+):([0-9]+)
45          :([0-9]+).([0-9]+):INFO:oejs.
46          AbstractConnector:Started
47          SelectChannelConnector@0.0.0.0:9080.*");
48
49     // Initializing variables
50     int line_num = 1;
51     boost::posix_time::ptime t;
52     boost::posix_time::ptime bootEndTime;
53     boost::gregorian::date date;
54     boost::smatch sm;

```

```

47     boost::gregorian::date tempD;
48     bool repeat = false;
49
50     while (std::getline(file, line)) {
51         if (boost::regex_match(line, sm, startup))
52         {
53             // checking for fail boot up
54             if (repeat) {
55                 output_file << "**** Incomplete
56                     boot ****\n" << std::endl;
57             }
58             output_file << "=== Device Boot ==="
59                 << std::endl;
60             output_file << line_num << "(" <<
61                 argv[1] << "): ";
62             output_file << (sm[1]) << '-' << (sm
63                 [2]) << '-' << (sm[3])
64             << ' ' << (sm[4]) << ':' << (sm[5])
65             << ':' << (sm[6])
66             << ' ' << "Boot Start" << std::endl;
67             tempD = boost::gregorian::
68                 from_un delimited_string(sm[1]+sm[2]+
69                     sm[3]);
70             date = tempD;
71             boost::posix_time::ptime tempT =
72                 boost::posix_time::from_iso_string
73                 (sm[1]+sm[2]+sm[3]+'T'+sm[4] + sm[5]
74                     + sm[6]);
75             t = tempT;
76             repeat = true;
77
78             // checking for successful bootup
79         } else if (boost::regex_match(line, sm,
80             success)) {
81             output_file << line_num << "(" <<
82                 argv[1] << "): ";
83             output_file << (sm[1]) << '-' << (sm
84                 [2]) << '-' << (sm[3])
85             << ' ' << (sm[4]) << ':' << (sm[5])
86             << ':' << (sm[6])

```

```

75         << ' ' << "Boot Completed" << std::
            endl;
76         tempD = boost::gregorian::
77         from_undelimited_string(sm[1]+sm[2]+
            sm[3]);
78         date = tempD;
79         boost::posix_time::ptime tempT =
80         boost::posix_time::from_iso_string
81         (sm[1]+sm[2]+sm[3]+'T'+sm[4] + sm[5]
            + sm[6]);
82         bootEndTime = tempT;
83         boost::posix_time::time_duration td =
            bootEndTime-t;
84         output_file << "\tBoot Time: " << td.
            total_milliseconds()
85         << "ms" << std::endl << std::endl;
86         repeat = false;
87     }
88     line_num += 1;
89 }
90 return 0;
91 }

```